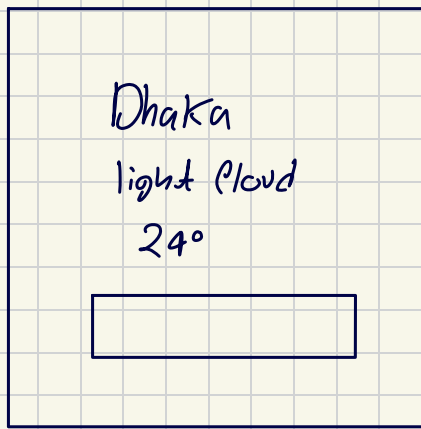



Quiz 01 - (4.00 - 4.30) 29/5/25

* Topics All topics up to today's class

Topics:

- * Cross platform / Native Platform
- * Why React is better than vanilla JS.
- * JSX
- * Props
- * States
- * Callback Functions
- * Asynchronous Function
- * Await
- * Conditional Rendering
- * Lifecycle Method
- * Basic concepts of JS.



* Connect API to
Fetch data

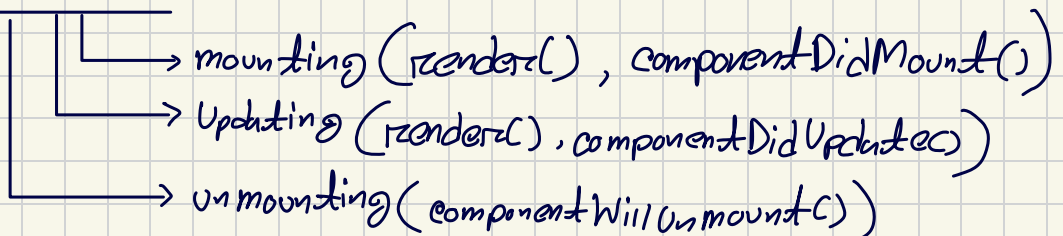
* Utils/api.js

await → Execution will be paused until we get
the data from API.

Lifecycle Methods:

Lifecycle methods are the methods that are
part of class components and allow you to write
codes in specific points of component's lifetime.

Lifetime



Conditional Rendering: Responding to invalid inputs.

{ `<div>` && `</div>` } → as this is an JS code
we write this inside
of curly braces

JS

```
console.log("Hello World")
```

To declare Variables:

Let

Var

Const

```
let x = 5
```

```
console.log(typeof(x))
```

data type

→ String

→ Number

→ BigInt

→ Boolean

→ Undefined

→ Object

→ Null

let = 123 56 789 ----- 123 n
(BigInt)

Let vs Var

let = is Block-Scoped (This block can be the full file if we declare it in the file outside of all functions)

var = is Function-Scoped

let x=5

let y=5

console.log(x+y)

output: 55

console.log(x==y)

output: true

== (lose equality)

(value same)

console.log(x===y)

output: false

=== (strict equality)

(value and type same)

Const: When we will not update the variable.

```
const cash = 90
```

```
const entry = (cash > 100) ? "Allowed" : "Not Allowed";
```

```
console.log(entry)
```

output: Not Allowed

Object

```
const Person =
```

```
{  
  firstName = "Shufan",  
  lastName = "Shahi",  
};
```

```
console.log(Person.FirstName);
```

or

```
console.log(Person["FirstName"]);
```

```
const fruit = ["Apple", "Banana", "Orange"]
```

```
console.log(fruit[0]);
```

```
const Person =
```

```
{  
  FirstName = "Shufan",  
  LastName = "Shahi"
```

```
  fullName: function() {
```

```
    return this.FirstName + " " +  
           this.LastName
```

```
  }
```

```
};
```

console.log(Person.FullName())

```
{! error && {  
  view: {  
    <Text> {location} </Text>  
    <Text> {weather} </Text>  
    <Text> {temp} </Text>  
  }  
}
```

let
var
const

data types:
string, number, bigint, Boolean, Undefined,
Object, Null

let x = ...;
console.log(typeof(x));

string = 'abcd' // '2';

number = 123;

BigInt = 123456789n; // otherwise number

Boolean = true/false

Undefined: let x;

Object: let x = null;

function → keyword needed.

function f1()

```
{  
  for (let i = 0; i < 5; i++)
```

```
{  
  console.log(i);
```

```
}  
console.log(i); // error if i = let,  
no if i = var.
```

function f1();

output 0 1 2 3 4

let → block scoped.

will work only inside the
block it was defined.

var can be accessed outside block
& is function scoped.

```

var v1 = 10;
let l2 = 31;

function f1()
{
  var v0 = 9;
  let l0 = 8;

  if (1)
  {
    let l1 = 7;
    var v3 = 121;

    console.log(l0);
    console.log(l1);
    console.log(v1);
    console.log(l2);
  }

  console.log(l0); // 11??
  console.log(v3);
  console.log(v1);
  console.log(l2);
}

```

f1()

Output: 8 7 10 31 8 121 10 31

⊗ Defining an object:

```

function f1()
{

```

const person =

```

{
  first name: "Gieralt";
  last name: "Of Rivia";
};

```

```

full name: function() {
  return this.firstname + " " + this.lastname;
}

```

```

console.log(person.firstname); // Gieralt
console.log(person["firstname"]); // Gieralt
console.log(person.fullname()); // Gieralt of Rivia
const fruits = ["Apple", "Banana", "Orange"];
console.log(fruits[0]); // Apple

```

function f1()

```

{
  let x = 5;
  let y = '5';
  let z = '6';
  console.log(x+y);
}

```

const carh = 10;
const entrv = (carh > 100)?
"Allowed" : "Not allowed";
console.log(entrv);

f1(); output: 55 not allowed.

number + string ⇒ string + string
 '5' + '5' ⇒ 55
 5 + '5' ⇒ 55

(==) ⇒ loose equal, checks only value, not type.
 here: x == y returns true
 (===) ⇒ strict equal, checks both type & value
 here: x === y returns false